Ders 10 Genişletilmiş Notu: Öneri Sistemleri: İçerik Tabanlı Öneri ve İşbirlikçi Filtreleme

Ders 10 Genişletilmiş Notu: Öneri Sistemleri: İçerik Tabanlı Öneri ve İşbirlikçi Filtreleme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Öneri Sistemlerine Giriş
- 2. Uzun Kuyruk Olgusu
- 3. Öneri Türleri
- 4. Fayda Fonksiyonu ve Fayda Matrisi
- 5. Öneri Sistemlerinin Üç Temel Sorunu
- 6. Puanları Toplamak: Açık ve Örtük Puanlar
- 7. Soğuk Başlangıç Sorunu
- 8. İçerik Tabanlı Öneri
- 9. İşbirlikçi Filtreleme
- 10. Kullanıcı-Kullanıcı ve Öğe-Öğe İşbirlikçi Filtreleme
- 11. Öneri Algoritmalarını Değerlendirmek
- Örnek Sorular

1. Öneri Sistemlerine Giriş

Hoca bu derste kitabın 9. bölümüne, şimdiye kadar hiç görmediğimiz öneri sistemleri (recommendation systems) konusuna geçti. Ders büyük ölçüde İngilizce anlatıldı.

Hoca konuya Web'deki uygulamaları sayarak başladı. Kitap satın alınabilen bulut tabanlı uygulamalar, film ve dizi izlenen Netflix, kısa videoların izlendiği YouTube, müzik dinlenen Spotify ve aranıp gezilen çok sayıda belgeden oluşan Web'in kendisi. Bu sitelere gittiğimizde genelde arama yaparız. Bunun iki yolu vardır:

- **Gezinme (browsing):** Aradığımız öğenin kategorisine gidip orada ararız.
- **Arama kutusu:** Sorgumuzu metin kutusuna yazarız.

Milyonlarca öğe olan bir sitede az bilinen bir öğeyi bu iki yolla bulmak verimli değildir, çok fazla zaman alır. Bu yüzden e-ticaret sitelerinin kullanıcılara öğe önerecek algoritmalara ihtiyacı vardır.

2. Uzun Kuyruk Olgusu

Hoca fiziksel mağaza ile çevrimiçi mağaza arasındaki farkı anlattı. Sanal marketlerde (virtual markets) satılabilecek ürün sayısı sınırsızdır. Çünkü bir ürünü e-ticaret sitesine koymanın maliyeti neredeyse sıfırdır. Yapılması gereken tek şey ürünü veritabanına eklemektir. Fiziksel bir mağaza ise satmak istediği ürün sayısını artırmak için fiziksel alanını büyütmek, daha çok raf koymak zorundadır. Bu yüzden dijital mağazalarda satılan öğe sayısı büyür. Bu olguya uzun kuyruk (long tail) denir.

2.1 Uzun Kuyruk Grafiği

Slayttaki grafikte:

- **x ekseni:** Popülerliğe göre sıralanmış öğeler. En popüler öğe en baştadır, ikinci öğe daha az popülerdir ve böyle devam eder.
- **y ekseni:** Haftalık satın alma sayısı. Yani aslında popülerlik.

  Ders sonrası not: Hoca x ekseni değerleri arttıkça öğelerin popülerliğinin "arttığını" söyledi. Öğeler popülerliğe göre azalan sırada dizildiği için doğrusu, x arttıkça popülerliğin azalmasıdır. Hocanın "birinci öğe çok popüler, ikinci öğe daha az popüler" açıklaması da bu doğru yorumla tutarlıdır.

Grafikteki mavi çizgi bir popülerlik eşiğidir. Fiziksel mağaza yalnız bu eşiğin üstündeki çok popüler öğeleri satabilir. Hocanın örneği: Kadıköy'deki bir D&R şubesinin müdürü olduğunuzu düşünün. Mağazanızın belirli bir alanı vardır. Bu alana ancak satılma olasılığı yüksek, çok popüler ürünleri koyarsınız.

Aynı mağazanın dijital sürümünde, örneğin dijital D&R ya da Migros Sanal Market'te, satılma olasılığı düşük, az popüler öğeleri de bulabilirsiniz. Bu mağazalara konabilecek öğe sayısı sınırsızdır, ama bu öğelerin popülerliği düşüktür. Eşiğin altında kalan alana bakınca, dijital mağazaya konabilecek ürün sayısının fiziksel mağazadakinden çok daha fazla olduğu görülür.

Hocanın tanımıyla uzun kuyruk, az popüler öğeleri çevrimiçi mağazalara sınırsızca koyabilmektir. Böylece öğenin kısa sürede satılıp satılmayacağını önemsemezsiniz, ama bu az popüler öğelere her zaman erişim sağlamış olursunuz.

2.2 Uzun Kuyruğun Sorunu

Uzun kuyruğun sorunu şudur: Mağazaya çok fazla öğe koyduğunuzda son kullanıcının bu öğeleri bulması zorlaşır. Çünkü öğelere erişmenin tek yolu arama kutusu ya da kategorilerdir. Amazon'da milyonlarca öğe varsa, az popüler bir öğeyi arayan kullanıcı onu kolay bulamaz. Öneri sistemi, az popüler öğeleri son kullanıcıya getirerek aradığını bulmasını sağlar.

> **Ders sonrası not:** Hoca 00:30:41'de "fiziksel mağazanıza çok fazla öğe koyarsanız" dedi. Bağlamdan anlaşıldığı gibi burada kastedilen çevrimiçi mağazadır.

2.3 Touching the Void Örneği

Hoca öneri sistemlerinin etkisini bir kitap örneğiyle anlattı. "Touching the Void" dağcılık üzerine bir kitaptır. Aynı konudaki "Into Thin Air" kitabından önce yayımlanmıştır. İlk kitap yayımlandığında pek popüler olmamıştı. Yıllar sonra "Into Thin Air" çok satınca, öneri motorları "Into Thin Air" alanlara "Touching the Void" kitabını önerdi. Bir süre sonra "Touching the Void" da çok satanlar listesine girdi ve "Into Thin Air"den daha çok satmaya başladı. Yani bir kitap, öneri motoru sayesinde başka bir kitabı çok satan yaptı. Bu, öneri motorunun insanların satın alma davranışını nasıl etkileyebileceğinin iyi bir örneğidir.

> **Ders sonrası not:** Hoca ilk kitabın ikinciden "bir iki yıl önce" yayımlandığını söyledi. "Touching the Void" büyük olasılıkla 1988'de, "Into Thin Air" ise 1997'de yayımlandı; aradaki fark büyük olasılıkla yaklaşık dokuz yıldır.

Öneri motorları farklı platformlarda kullanılır: D&R gibi kitap satış siteleri, Netflix gibi film siteleri, Spotify gibi müzik mağazaları, Google News gibi haber siteleri. Facebook, LinkedIn ve Twitter da öneri motoru kullanır.

3. Öneri Türleri

Hoca öneri yapmanın üç yolunu saydı.

- **Editoryal ve elle hazırlanmış (editorial, hand-curated) öneriler:** Starbucks'a gittiğinizde genelde bir kara tahta görürsünüz. Çalışanlar önerdikleri içecekleri oraya yazar. Fiziksel bir D&R mağazasında bir köşeye birlikte dizilmiş kitaplar da D&R'ın kendi önerisidir. Bir gazetenin sitesinde gazetenin önerdiği makaleler de bu türdendir. Bunlar mağaza sahibinin ya da editörün yaptığı önerilerdir.

- **İstatistiğe dayalı öneriler:** Mağazada en çok satan 10 kitap. Spotify'ın "en çok dinlenen 10 şarkı" listesi. Netflix'in "geçen ay en çok izlenen filmler" listesi. Bunlar insanların ürünleri nasıl kullandığına dair istatistiklere dayanır.
- **Kişiye özel (tailored to individual users) öneriler:** Öneri algoritması kullanıcının e-ticaret sitesindeki dijital izlerini analiz eder: hangi kitapları okuduğunu, hangi filmleri izlediğini, hangi müzikleri dinlediğini. Kullanıcının geçmişini analiz ederek ona belirli bir öğe önerir.

Bu derste üçüncü türle ilgileniyoruz. Soru şudur: Bir e-ticaret sitesinde kullanıcı davranışını nasıl analiz eder, anlar, modelleriz ve sonra o kullanıcıya nasıl öğe öneririz?

4. Fayda Fonksiyonu ve Fayda Matrisi

Hoca problemi biçimsel olarak tanımladı. C müşteriler (customers) kümesi, S öğeler kümesidir. Öneri motoruyla yapmaya çalıştığımız şey, her müşterinin henüz görmediği öğelere vereceği puanı bulmaktır. Puan genelde 0 ile 5 arası yıldızla ya da 0 ile 1 arası bir gerçek sayıyla ölçülür.

Aradığımız fonksiyona fayda fonksiyonu (utility function) denir. Müşteri kimliğini ve öğe kimliğini girdi olarak alır ve bir puan üretir:

$$u : C \times S \to R$$

Burada R puanlar kümesidir. Bir müşteri için mağazadaki her öğenin puanını bulabilirsek öneri yapabiliriz. O kişi mağazamıza geldiğinde en yüksek puanlı öğeleri öneririz.

4.1 Fayda Matrisi

Fayda fonksiyonundan hesaplanan tabloya fayda matrisi (utility matrix) denir. Slayttaki örnekte dört kullanıcı (Alice, Bob, Carol, David) ve dört film vardı. Matristeki puanlar açık puanlardır (explicit ratings). Yani kullanıcı bu puanları kendisi vermiştir. Bir yayın servisinde film bitince yıldızlar çıkar ve servis sizden puan ister. Yıldızlara tıklayarak açık puan vermiş olursunuz.

Örnekte Alice iki filme, Bob iki filme, Carol iki filme, David bir filme puan vermişti. Bizi ilgilendiren bilinmeyen puanlardır. Örneğin Alice "Lord of the Rings" filmini izleseydi kaç verirdi? "Pirates" filmini izleseydi kaç verirdi? Diğer kullanıcılar için de aynı soruları sorarız. Bilinmeyen puanları hesaplarsak, Alice mağazaya geldiğinde bu iki filmden hangisine daha yüksek puan vereceğini biliriz ve onu öneririz: "Bu filmi beğenebileceğinizi düşünüyoruz."

5. Öneri Sistemlerinin Üç Temel Sorunu

Öneri algoritması tasarlamak zordur, çünkü üç sorun vardır.

1. **Bilinen puanları toplamak:** Öneri yapabilmek için kişinin geçmişini bilmemiz gerekir. Kişi satın alma davranışı ya da film ve müzik tercihleri hakkında bize bir geçmiş sunmalıdır. Örneğin Alice'in Avatar'ı izledikten sonra kaç puan verdiğini nasıl öğreneceğiz? Bu zordur, çünkü film izledikten ya da kitap okuduktan sonra az kişi görüş bildirir. Yalnız küçük bir kullanıcı grubu geri bildirim verir. Bu çözülmesi gereken temel bir zorluktur.
2. **Bilinmeyen puanları tahmin etmek:** Bilinen puanları bir şekilde elde ettikten sonra, onları kullanarak bilinmeyenleri hangi algoritmalarla bulacağız?
3. **Başarıyı ölçmek:** Bilinmeyen puanları hesaplayıp öneri yaptıktan sonra, öneri algoritmasının başarısını nasıl ölçeceğiz? Öneri motorunun performansını değerlendirmek için iyi bir yöntem nedir?

6. Puanları Toplamak: Açık ve Örtük Puanlar

Puan toplamanın iki yolu vardır.

6.1 Açık Puanlar

Kullanıcıdan izledikten sonra görüşünü doğrudan girmesi istenir. Netflix'te film bitince yıldızlar gösterilir ve kullanıcıdan görüşünü yıldızla girmesi istenir. D&R olsaydınız, kitap satın alan birine e-posta gönderip kitap hakkında ne düşündüğünü sorabilirdiniz. Ama bu yöntem ölçeklenmez (doesn't scale), çünkü yalnız az sayıda kullanıcı görüş bildirir. Bu yüzden başka yollar aramak gerekir.

6.2 Örtük Puanlar

Kullanıcının görüşünü dolaylı olarak, başka ölçütlere bakarak çıkarırız (implicit ratings).

- Biri bir filmi birkaç kez izlediyse, o filmi beğenmiş olmalıdır. Bunun için o filmin izlenme sayısına bakılabilir.
- Aynı şey müzik için de geçerlidir. Biri Spotify'da bir şarkıyı çok kez dinliyorsa, o şarkıyı çok sevdiğini düşünebiliriz.
- Bir e-ticaret sitesinde biri bir ürünün detay sayfasına farklı oturumlarda, farklı günlerde defalarca bakıyorsa, o ürünle çok ilgilendiğini söyleyebiliriz. Bu kişi o ürüne yüksek puan verirdi.

Yani kişinin öğeyi izleme, dinleme ya da satın alma sıklığına göre o öğeye verdiği puan değerini artırabiliriz. Kişinin öğeyle etkileşim sıklığından hesaplanan bu puana örtük puan denir. Açık puanın alınamadığı durumda kullanılır.

7. Soğuk Başlangıç Sorunu

Bugünkü dersin odağı ikinci sorundur: bilinmeyen puanları hesaplamak. Bunun için iki algoritma öğreneceğiz: içerik tabanlı (content-based) algoritmalar ve işbirlikçi (collaborative) algoritmalar. Başka algoritmalar da vardır, ama ders bu ikisine odaklanıyor.

Bu algoritmaların tek sorunu, kullanıcının e-ticaret sitesindeki geçmişine ihtiyaç duymalarıdır. Mağazaya yeni bir kullanıcı gelirse onun satın alma davranışı, müzik dinleme ya da film izleme geçmişi hakkında hiçbir bilgimiz yoktur. Bu durumda ona film, müzik ya da kitap öneremeyiz. Benzer şekilde mağazaya yeni bir öğe gelirse ve bu öğeye bağlı bir geçmiş yoksa, onu kullanıcılara önermek zor olur. Bu soruna soğuk başlangıç sorunu (cold start problem) denir. Öğreneceğimiz algoritmalar ancak veritabanında belirli bir geçmiş verisi biriktikten sonra verimli çalışır.

8. İçerik Tabanlı Öneri

8.1 Ana Fikir

İçerik tabanlı öneri algoritması (content-based recommendation), bir müşteriye o müşterinin daha önce puanladığı öğelere dayanarak öğe önerir. Öğeler film, web sitesi, blog, hatta insan olabilir. Farklı e-ticaret mağazalarında ya da sosyal ağlarda her tür öğe olabilir.

Slayttaki resimde bir kişi belirli öğeleri satın alıyor ya da beğeniyor. Algoritmanın adımları şöyledir:

1. Kullanıcının beğendiği her öğe için bir özellik vektörü (feature vector) oluşturulur. Öğe bir filmse özellikler filmin yönetmeni, filmdeki oyuncular, filmin türü (aksiyon mu, bilim kurgu mu) gibi şeylerdir. Bu vektöre öğe profili vektörü (item profile vector) denir.
2. Kullanıcının beğendiği öğelerin profil vektörlerinin ortalaması alınır. Böylece bütün bu öğeleri temsil eden tek bir vektör elde edilir. Buna kullanıcı profili vektörü (user profile vector) denir. Bu vektör kullanıcının önceden tercih ettiği öğeleri temsil eder.
3. Kullanıcı profili vektörü, veritabanındaki diğer bütün öğe profili vektörleriyle karşılaştırılır. Her özellik değeri bir gerçek sayı gibi düşünülür ve iki vektör arasındaki uzaklığa bakılır.
4. Uzaklık küçükse, o öğe kullanıcıya önerilebilecek öğelerden biridir. Kullanıcı profiline yakın olanlar önerilir.

8.2 Metin Öğeleri İçin Özellikler: TF-IDF

Öğe bir makale ya da haber gibi bir metinse, onu temsil edecek özellikler kelimeler olabilir. Bunlar makaleden çıkarılan kelimelerdir. Ama makaledeki her kelimeyi seçemeyiz, önemli kelimeleri seçmemiz gerekir. Bunun için önceki derslerde öğrendiğimiz TF-IDF ölçüsüne bakarız. TF-IDF iki alt ölçüden oluşur.

- **Terim sıklığı (term frequency, TF):** Bir kelimenin belirli bir makaledeki önemini gösterir. Kelime makalede çok tekrar ediyorsa, yani o makalede geçme sıklığı yüksekse, TF değeri yüksek olur. Bu da kelimenin o makalede önemli olduğunu söyler.
- **Ters belge sıklığı (inverse document frequency, IDF):** Bir kelimenin bir belge koleksiyonundaki değerini gösterir. Örneğin 100 belgeden oluşan bir koleksiyonunuz olsun. Kelime koleksiyondaki her belgede geçiyorsa, her yerde olduğu için o koleksiyondaki değeri yüksek değildir. Kelime koleksiyonda yalnız birkaç belgede geçiyorsa, nadir bulunduğu için değeri yüksektir.

  Hocanın vurgusu: "Bir şey her yerdeyse değeri düşüktür. Bir şey nadir bulunuyorsa değeri yüksektir. Bunu böyle ezberleyebilirsiniz."

Slayttaki formülde n_i, i. kelimeyi içeren belge sayısıdır. Kelime çok belgede geçiyorsa n_i büyük, IDF küçük olur. Kelime az belgede geçiyorsa n_i küçük, IDF büyük olur:

$$IDF_i = \log \frac{N}{n_i}$$

Burada N koleksiyondaki toplam belge sayısıdır. Hocanın 100 belge örneğinde kelime 100 belgenin hepsinde geçerse $IDF = \log(100/100) = \log 1 = 0$ olur (ara adım eklendi). Yani her yerde geçen kelimenin koleksiyon içindeki değeri sıfırdır.

Bir koleksiyon içindeki bir belgede bir kelimenin değerini hesaplarken, o kelimenin o belgedeki TF değeri ile IDF değeri çarpılır:

$$w_{ij} = TF_{ij} \times IDF_i$$

Hoca bunu iki kez tekrar etti: TF bir kelimenin bir belge içindeki değerini, IDF bir kelimenin bir belge koleksiyonu içindeki değerini gösterir. Bulunan ağırlık, kelimenin o belgedeki önemini verir. TF-IDF puanlarıyla önemli kelimeler belirlenir ve bu kelimeler makalenin özellik vektöründe kullanılır.

> **Ek bilgi (derste anlatılmadı):** Kitapta (Mining of Massive Datasets) terim sıklığı büyük olasılıkla $TF_{ij} = f_{ij} / \max_k f_{kj}$ biçiminde tanımlanır. Burada f_{ij}, i. kelimenin j. belgede geçme sayısıdır; belgedeki en sık kelimeye bölmek, uzun belgelerin kelimelerine haksız avantaj vermeyi önler. Kitapta IDF için logaritmanın tabanı büyük olasılıkla 2 alınır. Bir belgenin profili olarak en yüksek TF-IDF puanlı kelimeler seçilir.

8.3 Kullanıcı Profili: Basit ve Ağırlıklı Ortalama

Kullanıcı profili vektörü, öğe profili vektörlerinin ortalamasıdır. Bu ortalama farklı biçimlerde alınabilir: basit ortalama (simple average) ya da öğelerin ağırlıklı ortalaması (weighted average). Bunun başka varyasyonları da vardır.

Oyuncu örneği: Boole profil Öğeler film, özellikler oyuncular olsun. Öğe profili vektöründe her özellik bir oyuncudur. Oyuncu o filmde oynuyorsa değer 1, oynamıyorsa 0'dır.

Verilen: Beş film ve iki oyuncu var. A oyuncusu iki filmde, B oyuncusu üç filmde oynuyor. Kullanıcı X bu beş filmin hepsini izlemiş.

Kullanıcı profilini hesaplamanın bir yolu basit ortalamadır. Beş öğe profili vektörünün önce birinci elemanlarının ortalaması alınır, sonra ikinci elemanlarının, sonra üçüncü elemanlarının ve böyle devam eder. Sonuç kullanıcı profili vektörüdür.

Diğer yol ağırlıklı ortalamadır. A oyuncusu beş filmin ikisinde oynamıştır, B oyuncusu beş filmin üçünde:

$$w_A = \frac{2}{5} = 0{,}4 \qquad w_B = \frac{3}{5} = 0{,}6$$

Bu ağırlıklar kullanıcı profilini hesaplarken her özellik, yani her oyuncu için kullanılır.

Oyuncu örneği: Yıldız puanlı, normalize profil Aynı örnekte bu kez 0-1 değerleri yerine 1-5 arası yıldız puanları var. Kullanıcı A oyuncusunun oynadığı iki filme 3 ve 5, B oyuncusunun oynadığı üç filme 1, 2 ve 4 vermiş. Bu kez ağırlıkları normalize ederek hesaplarız. Önceki sürümde ağırlıklar normalize edilmemişti.

Adım 1: Kullanıcının ortalama puanı. Kullanıcı beş puan vermiş. Toplamları:

$$3 + 5 + 1 + 2 + 4 = 15 \qquad \bar{r}_X = \frac{15}{5} = 3$$

Yani bu kişi bir filme ortalama 3 puan veriyor.

> **Ders sonrası not:** Hoca 01:27:31'de önce "kullanıcı ortalaması 5 olur" dedi, hemen ardından toplamı 15, puan sayısını 5 bularak ortalamanın 3 olduğunu hesapladı. Doğrusu 3'tür.

Adım 2: Ortalamayı çıkarmak. Her puandan 3 çıkarılır. Bunlar normalize puanlardır.

- A'nın filmleri: $3 - 3 = 0$ ve $5 - 3 = 2$.
- B'nin filmleri: $1 - 3 = -2$, $2 - 3 = -1$ ve $4 - 3 = 1$.

Adım 3: Profil ağırlıkları. Her oyuncu için normalize puanların ortalaması alınır (ara adım eklendi; hoca yalnız "A için ve B için ortalama ağırlığı hesaplarım" dedi, sayıları altyazıda okumadı):

$$w_A = \frac{0 + 2}{2} = 1 \qquad w_B = \frac{-2 - 1 + 1}{3} = -\frac{2}{3}$$

Bu profil ağırlıklarıyla öğe vektörlerinden kullanıcı profili vektörü ağırlıklı ortalamayla hesaplanır. Böylece kullanıcı profilinin basit ortalama, ağırlıklı ortalama ve normalize ağırlıklı ortalamayla nasıl hesaplanacağı görülmüş oldu.

8.4 Profilleri Karşılaştırmak: Kosinüs Benzerliği

Kullanıcı profili hesaplandıktan sonra veritabanındaki her öğe profili vektörüyle karşılaştırılır. İki vektör arasındaki uzaklık küçükse, o öğe profili kullanıcı profiline benziyordur ve önerilebilir. İki vektör için kosinüs benzerliği (cosine similarity) kullanılabilir: İki vektör arasındaki açıya bakılır ve açının kosinüsü alınır.

$$\cos(x, y) = \frac{x \cdot y}{\|x\| \, \|y\|}$$

> **Ders sonrası not:** Hoca "kosinüs benzerliği küçükse iki vektör birbirine benzer, büyükse birbirinden uzaktır" dedi. Doğrusu tersidir. Açı küçükse kosinüs büyük olur ve vektörler benzerdir. Dersin ilerisindeki işbirlikçi filtreleme örneklerindeki sonuçlar ($sim(A, B) > sim(A, C)$, en yüksek benzerlikli satırları seçmek) bu doğru yorumla tutarlıdır.

8.5 İçerik Tabanlı Önerinin Artıları

- **Başka kullanıcıların verisine gerek yoktur.** Bana yalnız kendi verimi analiz ederek öneri yapabilirsiniz.
- **Kendine özgü zevki olan kullanıcılara öneri yapılabilir.** Örneğin hep klasik müzik dinleyen birine özel klasik müzik önerileri yapılabilir.
- **Popüler olmayan öğeler önerilebilir.** Mağazaya yeni gelen bir öğe de önerilebilir. Çünkü yalnız o öğenin özelliklerine, öğe profili vektörüne bakarız. Öğeye bağlı bir geçmişe ihtiyaç yoktur.
- **Sonuç açıkça açıklanabilir.** "Mağazada dinlediklerinizi, satın aldıklarınızı ya da izlediklerinizi analiz ederek bu öneriyi yapıyoruz" denebilir. Öneri kullanıcının kendi geçmişine dayandığı için nedenini açıklamak kolaydır.

8.6 İçerik Tabanlı Önerinin Eksileri

- **Her öğe için özellik vektörü bulmak zordur.** Film için yönetmen, oyuncular, tür gibi özellikler verdik. Peki bir görüntü için hangi özellikleri seçeceğiz? Müzik için en temsil edici özellikler nelerdir? Doğru ve uygun özellikleri bulmak zordur, ama çok önemlidir.
- **Aşırı uzmanlaşma (overspecialization):** Hep kişinin zaten bildiği türde ürünler önerilir. Hocanın örneği: "Ben klasik müzik dinleyicisiyim. Bana hep dinlediğim türde müzik önerirsiniz, kendi ilgi alanımın dışında hiçbir şey önermezsiniz." Oysa insanların birden çok ilgi alanı olabilir. Spotify'da klasik müzik dinlemem, başka türleri sevmediğim anlamına gelmez. Bu algoritma yalnız kullanıcı profilinin içinden öneri yapar, profilin dışına çıkmaz.
- **Soğuk başlangıç:** En önemli sorun budur. Henüz kullanıcı profili oluşmamış bir kullanıcıya öneri yapılamaz.

9. İşbirlikçi Filtreleme

9.1 Ana Fikir

İşbirlikçi filtreleme (collaborative filtering) algoritmasında bir kullanıcının tercihlerine bakılır ve bu tercihler diğer kullanıcıların tercihleriyle karşılaştırılır. Aynı tercihlere sahip kişiler bulunur. Sonra birinci kullanıcının görmediği, ama benzer ilgi alanlarına sahip kişilerin gördüğü öğeler birinci kullanıcıya önerilir.

Adımlar şöyledir:

1. Öneri yapmak istediğimiz kullanıcının tercih vektörü hesaplanır.
2. Bu vektör veritabanındaki diğer kullanıcıların tercih vektörleriyle karşılaştırılır.
3. Tercihleri benzer kullanıcılar bulunur.
4. X kişisinin görmediği, ama benzer tercihli kişilerin gördüğü öğeler X'e öneri olarak sunulur.

Hocanın örneği: "Ben bir Netflix kullanıcısıyım. Bilim kurgu ve tarih filmlerini seviyorum." Kullanıcı veritabanında benzer ilgi alanına sahip, yani bilim kurgu ve tarih filmlerini seven kişiler aranır. Sonra bu kişilerin tarih ve bilim kurgu dışında başka filmler izleyip izlemediğine bakılır. Örneğin bunlara ek olarak aksiyon filmlerini de seviyorlarsa, benzer tercihli kişiler aksiyon kategorisinde film izlediği için bana aksiyon filmi önerilir.

Bu algoritma "sürpriz"e dayanır. Beni şimdiye kadar görmediğim türde bir öğe önerisiyle şaşırtır. Ama aynı tercihlere sahip kişiler o filmi beğendiği için benim de beğenmem olasıdır.

9.2 Örnek Fayda Matrisi

Hoca işbirlikçi filtrelemenin nasıl uygulandığını dört kullanıcı ve yedi filmden oluşan bir fayda matrisiyle gösterdi. Puanlar açık geri bildirimlerdir, yani kullanıcıların verdiği yıldızlardır. Amaç yine bilinmeyen değerleri bulmaktır. Bunları hesaplarsak, örneğin A kullanıcısı Netflix'e geldiğinde izlemediği dört film arasından en yüksek puanlı olanı önerebiliriz.

Altyazıdan ve hocanın hesaplarından kurulan puanlar (D'nin puanları altyazıda okunmuyor):

Kullanıcı	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	4			5	1		
B	5	5	4				
C				2	4	5	

9.3 Sezgi: Hangi Kullanıcılar Benzer?

Hoca önce veriye bakıp gözlem yaptı.

- **A ve B:** İkisinin de puan verdiği tek bir ortak film var, Harry Potter 1. İkisi de yüksek puan vermiş. Yani ikisi de bu filmi beğenmiş.
- **A ve C:** İkisi de iki ortak filme puan vermiş: Twilight ve Star Wars 1. A Twilight'ı beğenmiş, Star Wars 1'i beğenmemiş. C Star Wars'u beğenmiş, Twilight'ı beğenmemiş. Bir filmi biri beğeniyor, öbürü beğenmiyor; diğer filmde de tersi. Yani A ve C'nin tercihleri tamamen zıt.

Bu yüzden tercih benzerliği açısından $sim(A, B) > sim(A, C)$ olmalıdır. Aradığımız, bu sezgiyi yakalayan bir uzaklık fonksiyonudur. Hoca şimdiye kadar öğrendiğimiz uzaklık fonksiyonlarını tek tek denedi.

9.4 Jaccard Benzerliği

İki vektör için Jaccard benzerliği, kesişimin birleşime bölünmesidir:

$$sim_J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

> **Ders sonrası not:** Hoca tanımı verirken "paydada kesişimi alırsınız, paydada birleşimi alırsınız" dedi. Doğrusu: payda (üstte) kesişim, paydada (altta) birleşim vardır.

A ve B: İkisinin birlikte izlediği film sayısı, yani kesişim 1'dir (HP1). İkisinin toplam izlediği farklı film sayısı, yani birleşim 5'tir (HP1, HP2, HP3, TW, SW1).

$$sim_J(A, B) = \frac{1}{5} = 0{,}2$$

A ve C: Birlikte izledikleri farklı film sayısı 4'tür (HP1, TW, SW1, SW2). İkisinin de izlediği filmler Twilight ve Star Wars 1'dir, yani kesişim 2'dir.

$$sim_J(A, C) = \frac{2}{4} = 0{,}5$$

Sonuç $sim_J(A, C) > sim_J(A, B)$ çıkar. Bu doğru değildir, sezgimizi yakalamıyor. Jaccard yalnız kimin hangi filmi izlediğine bakar, verilen puanları hesaba katmaz. Bu yüzden kullanıcı tercihlerinin benzerliğini bulmak için Jaccard benzerliğini kullanamayız.

9.5 Kosinüs Benzerliği

Kosinüs benzerliğini hesaplamak için her kullanıcıyı bir vektör olarak ele almalıyız. Bu yüzden bilinmeyen değerlere 0 konur. A ve B için bilinmeyenlere 0 koyup kosinüs formülünü uygulayınca:

$$sim(A, B) = \frac{4 \cdot 5}{\sqrt{4^2 + 5^2 + 1^2}\,\sqrt{5^2 + 5^2 + 4^2}} = \frac{20}{\sqrt{42}\,\sqrt{66}} \approx 0{,}38$$

$$sim(A, C) = \frac{5 \cdot 2 + 1 \cdot 4}{\sqrt{42}\,\sqrt{2^2 + 4^2 + 5^2}} = \frac{14}{\sqrt{42}\,\sqrt{45}} \approx 0{,}32$$

(Pay ve paydadaki ara adımlar eklendi; hoca yalnız sonuçları okudu.)

Bu kez $sim(A, B) > sim(A, C)$ çıktı. Ama hoca bunun da aradığımızı yakalamadığını söyledi. Çünkü iki benzerlik değeri birbirine çok yakın. Oysa A ve B çok benzer, A ve C ise neredeyse zıt; bu iki değerin birbirinden uzak olması gerekir.

> **Ek bilgi (derste anlatılmadı):** Kosinüs benzerliğinin burada zayıf kalmasının nedeni, bilinmeyen puanlara 0 konmasıdır. 0, 1-5 ölçeğinde "çok kötü puan" gibi davranır; yani izlenmemiş film, beğenilmemiş film gibi hesaba girer. Ayrıca bütün puanlar pozitif olduğu için zıt zevkler de pozitif benzerlik üretir. Merkezleme, ortalamanın altındaki puanları negatife çevirerek bu iki sorunu birlikte hafifletir.

9.6 Merkezlenmiş Kosinüs (Pearson Korelasyonu)

Çözüm, kullanıcıların puanlarını kendi satır ortalamalarıyla (row mean) normalize etmektir. Buna merkezlenmiş kosinüs (centered cosine) denir.

A kullanıcısı: A üç filme puan vermiş. Toplam puan:

$$4 + 5 + 1 = 10 \qquad \bar{r}_A = \frac{10}{3}$$

Bu A'nın satır ortalamasıdır. A'nın her puanından $10/3$ çıkarılır:

$$4 - \tfrac{10}{3} = \tfrac{2}{3} \qquad 5 - \tfrac{10}{3} = \tfrac{5}{3} \qquad 1 - \tfrac{10}{3} = -\tfrac{7}{3}$$

B kullanıcısı: B de üç filme puan vermiş. Toplam 14, ortalama $14/3$. Bu değer 5'ten, 5'ten ve 4'ten çıkarılır:

$$5 - \tfrac{14}{3} = \tfrac{1}{3} \qquad 5 - \tfrac{14}{3} = \tfrac{1}{3} \qquad 4 - \tfrac{14}{3} = -\tfrac{2}{3}$$

Aynı işlem C ve D için de yapılır. C için (ara adım eklendi): toplam $2 + 4 + 5 = 11$, ortalama $11/3$, normalize puanlar $-5/3$, $1/3$, $4/3$.

Artık fayda matrisindeki açık puanlar normalize puanlardır. Sonra kosinüs benzerliği yine bilinmeyenlere 0 konarak hesaplanır:

$$sim(A, B) = \frac{\frac{2}{3} \cdot \frac{1}{3}}{\sqrt{\frac{4+25+49}{9}}\sqrt{\frac{1+1+4}{9}}} = \frac{2}{\sqrt{78}\sqrt{6}} \approx 0{,}09$$

$$sim(A, C) = \frac{\frac{5}{3} \cdot (-\frac{5}{3}) + (-\frac{7}{3}) \cdot \frac{1}{3}}{\sqrt{\frac{78}{9}}\sqrt{\frac{25+1+16}{9}}} = \frac{-32}{\sqrt{78}\sqrt{42}} \approx -0{,}56$$

(Pay ve paydadaki ara adımlar eklendi; hoca sonuçları 0,09 ve -0,56 olarak okudu.)

Yine $sim(A, B) > sim(A, C)$, ama aradaki fark artık çok daha büyük. Normalizasyon yapmadığımızda iki benzerlik birbirine yakındı. Şimdi benzerlik farkını olması gerektiği gibi yakalayabiliyoruz. Hoca, kullanıcı tercih vektörlerinin benzerliğini hesaplamak için merkezlenmiş kosinüs benzerliğinin, diğer adıyla Pearson korelasyonu benzerliğinin (Pearson correlation similarity) iyi bir ölçü olduğunu söyledi.

> **Ek bilgi (derste anlatılmadı):** Merkezlenmiş kosinüste ortalamanın üstündeki puanlar pozitif, altındakiler negatif olur; bilinmeyen puana 0 konması artık "kullanıcının ortalaması kadar" anlamına gelir. Böylece hep yüksek puan veren "cömert" kullanıcılarla hep düşük puan veren "katı" kullanıcılar da karşılaştırılabilir hale gelir. Ortak puanlanan öğeler üzerinden hesaplanan klasik Pearson korelasyonuyla büyük olasılıkla aynı fikirdedir, ama tam formülü küçük farklar gösterebilir.

10. Kullanıcı-Kullanıcı ve Öğe-Öğe İşbirlikçi Filtreleme

İkinci aradan sonra hoca Pearson korelasyonunun bilinmeyen puanları bulmak için nasıl uygulanacağını bir örnek üzerinde gösterdi.

İşbirlikçi filtreleme iki biçimde yapılabilir. Yöntem aynıdır, tek fark fayda matrisinin yönüdür.

- **Kullanıcı-kullanıcı (user-user) işbirlikçi filtreleme:** Fayda matrisinin satırlarında kullanıcılar, sütunlarında öğeler vardır.
- **Öğe-öğe (item-item) işbirlikçi filtreleme:** Satırlarda öğeler (bu örnekte filmler), sütunlarda kullanıcılar vardır.

Hoca örneği öğe-öğe işbirlikçi filtrelemeyle gösterdi.

10.1 Komşuluk Parametresi

Her işbirlikçi filtrelemede bir komşuluk (neighborhood) değeri belirlenmelidir. Bu örnekte komşuluk değeri $|N| = 2$'dir. Bu parametre, bilinmeyen puanı bulurken en benzer kaç satırın hesaba katılacağını söyler.

> **Ders sonrası not:** Hoca komşuluk değerini "kaç benzer kullanıcıya bakılacağı" diye açıkladı. Öğe-öğe filtrelemede satırlar öğeler olduğu için burada bakılan, en benzer $|N|$ öğedir (filme). Hocanın örnekte seçtiği de iki filmdir (3. ve 6. satırlar).

10.2 Öğe-Öğe Tahmin Örneği

Hedef: 5. kullanıcının 1. filme vereceği bilinmeyen puan r_{15}.

Adım 1: En yakın iki filmi bulmak için satır ortalamaları. Her satırın (her filmin) ortalaması hesaplanır. Birinci satırın ortalaması 3,6'dır.

Adım 2: Normalizasyon. Her satırdaki her bilinen puandan o satırın ortalaması çıkarılır. Aynı şey 2., 3., 4., 5. ve 6. satırlar için yapılır. Böylece fayda matrisinin normalize sürümü elde edilir.

Adım 3: Benzerlikler. Normalize matriste her satır vektörünün 1. satır vektörüyle kosinüs benzerliği hesaplanır. Çünkü bilinmeyen değer 1. satırdadır. 1. satırın kendisiyle benzerliği 1'dir: İki vektör aynıysa benzerlik 1 olur.

Adım 4: Komşuları seçmek. Komşuluk değeri parametre olarak 2 verildiği için en yüksek benzerlikli iki satır seçilir: 3. ve 6. satırlar. Ayrıca bu satırlarda 5. kullanıcının puanı olduğundan emin olunur. Yani 5. kullanıcı 3. ve 6. filmlere puan vermiştir: 3. filme 2, 6. filme 3.

Adım 5: Ağırlıklı ortalama. 3. satırla 1. satırın benzerliği $s_{13} = 0{,}41$, 6. satırla 1. satırın benzerliği $s_{16} = 0{,}56$'dır. Benzerliklerle ağırlıklı ortalama alınır. Payda her benzerlik, kullanıcının o filme verdiği puanla çarpılır; paydada benzerlikler toplanır:

$$r_{xi} = \frac{\sum_{j \in N(i;x)} s_{ij} \cdot r_{xj}}{\sum_{j \in N(i;x)} s_{ij}}$$

$$r_{15} = \frac{0{,}41 \cdot 2 + 0{,}56 \cdot 3}{0{,}41 + 0{,}56} = \frac{0{,}82 + 1{,}68}{0{,}97} = \frac{2{,}50}{0{,}97} \approx 2{,}6$$

(Pay ve paydadaki ara toplamlar eklendi.)

> **Ders sonrası not:** Hoca 02:14:13'te "bu payda kısmı" dedi; bu kısım pay (üst) kısmıdır, ardından söylediği "paydada benzerlikleri topluyorum" doğrudur. Ayrıca hoca 6. satırın benzerliğini 0,56 olarak okudu. Slayttaki matris büyük olasılıkla kitabın ve Stanford ders slaytlarının 6 film × 12 kullanıcılı standart örneğidir; o matriste bu benzerlik büyük olasılıkla 0,59 çıkar (diğer benzerlikler 1,00; -0,18; 0,41; -0,10; -0,31). 0,59 ile tahmin $(0{,}82 + 1{,}77)/1{,}00 \approx 2{,}59$ olur; sonuç iki değerle de yaklaşık 2,6'dır.

Sonra: Aynı yolla fayda matrisindeki bütün bilinmeyen değerler, yani bütün beyaz kareler hesaplanır. 5. kullanıcı film mağazasına geldiğinde, izlediği filmler dışında kalan izlemediği filmlerden tahmini puanı en yüksek olan ona önerilir.

10.3 Neden Öğe-Öğe Daha İyi Çalışır?

İki yöntem de çalışır, ama uygulamada öğe-öğe işbirlikçi filtreleme, kullanıcı-kullanıcı işbirlikçi filtrelemeden daha iyi sonuç verir.

> **Hocanın vurgusu:** Uygulamada öğe-öğe işbirlikçi filtreleme, kullanıcı-kullanıcıdan çok daha iyi çalışır.

Kullanıcı-kullanıcı filtrelemede kullanıcıların satır vektörleri karşılaştırılır. Ama kullanıcıların tercihleri birçok farklı yönde değişebilir. Hocanın örneği: "Klasik müziği sevebilirim, aynı zamanda rock müziği de sevebilirim. Tamamen farklı tercihlerim olabilir." İnsanların tercihleri karmaşık olduğu için iki farklı kişiyi karşılaştırdığınızda kullanıcı tabanlı filtreleme tercih benzerliğini yakalayamayabilir. Öğe-öğe filtreleme ise bunu çok daha iyi yakalar.

> **Ek bilgi (derste anlatılmadı):** Kitaptaki gerekçe büyük olasılıkla şudur: Öğeler kullanıcılardan daha basit varlıklardır. Bir film genelde tek bir türe aittir, ama bir kullanıcı birbirinden çok farklı türleri sevebilir. Bu yüzden aynı türdeki iki öğenin benzerliği, iki kullanıcının benzerliğinden daha anlamlı ve daha kararlıdır.

11. Öneri Algoritmalarını Değerlendirmek

Hoca başta saydığı üç sorunu hatırlattı. Birinci sorun bilinen puanları toplamaktı. İkinci sorun öneri algoritmalarıyla bilinmeyen değerleri bulmaktı; bunun için iki yöntem öğrendik: içerik tabanlı filtreleme ve işbirlikçi filtreleme (kullanıcı-kullanıcı ve öğe-öğe). Üçüncü sorun, bilinmeyen değerleri bulduktan sonra öneri algoritmasının performansını nasıl anlayacağımızdır.

Slayttaki fayda matrisinde satırlar kullanıcılar, sütunlar öğeler (filmler) ve hücreler bilinen puanlardır. İçerik tabanlı algoritmayla ya da kullanıcı-kullanıcı işbirlikçi filtrelemeyle bilinmeyen değerler hesaplanabilir. Sonra algoritmanın performansını değerlendirecek bir yöntem gerekir.

11.1 Anket

Bir yol anket (survey) yapmaktır. Kullanıcılardan önerilerin sonucunu değerlendirmeleri istenir: "Size beş film öneriyorum. Önerilerim mantıklı mı? Beğendiniz mi?" Ama kullanıcıdan geri bildirim almak çok zordur. Bu yüzden önerilerin ne kadar başarılı olduğunu söyleyecek başka yöntemler gerekir.

11.2 Test Verisi Yöntemi

1. Fayda matrisinin küçük bir kısmı test verisi olarak belirlenir. Slayttaki örnekte bu bölgede beş bilinen puan vardı.
2. Bu beş puanın değerleri bilinir, ama test verisi için bilinmiyormuş gibi davranılır. Bu puanların yerine soru işareti konur.
3. Algoritma çalıştırılır ve soru işaretleri dahil bütün bilinmeyen değerler bulunur.
4. Test verisinde artık hem bilinen gerçek değerler (ground truth) hem de hesaplanan puanlar vardır. İkisi bir hata ölçüsüyle karşılaştırılır.

Hata ölçüsü olarak kök ortalama kare hata (root mean square error, RMSE) kullanılabilir. RMSE gerçek puan ile tahmin edilen puan arasındaki farkı söyler:

$$RMSE = \sqrt{\frac{1}{N}\sum_{xi}\left(r_{xi} - r^*_{xi}\right)^2}$$

Burada r_{xi} tahmin edilen, r^*_{xi} gerçek puan, N test verisindeki puan sayısıdır. RMSE küçükse öneri algoritması iyi öneriler yapıyordur. RMSE büyükse algoritmanın iyileştirilmesi (optimization) gerekir.

RMSE hata hesaplama yöntemlerinden yalnız biridir. Hoca başka sürümler olarak ortalama mutlak hata ve ortalama mutlak yüzde hatayı saydı. Değerlendirmenin diğer alternatifi kullanıcı anketleridir: Bu durumda öneri algoritmasının başarısını hesaplamak için kullanıcıların değerlendirmeleri alınır.

> **Ders sonrası not:** Hoca bu ölçüleri "mean average error" ve "mean average percentage error" diye adlandırdı. Doğru adlar "mean absolute error" (MAE) ve "mean absolute percentage error" (MAPE)'dir.

Örnek Sorular

Bu derste öğrencilerden akademik bir soru gelmedi. Aşağıdaki sorular hocanın ders içinde sınıfa sorup kendisinin cevapladığı sorulardır.

Soru 1 (derste sözlü, 01:26:56): Kullanıcı X beş film izlemiş. A oyuncusunun oynadığı iki filme 3 ve 5, B oyuncusunun oynadığı üç filme 1, 2 ve 4 vermiş. Kullanıcının ortalama puanı nedir ve normalize profil ağırlıkları ne olur?

Derste verilen çözüm: Puan sayısı 5, toplam $3 + 5 + 1 + 2 + 4 = 15$, ortalama $15/5 = 3$. Her puandan 3 çıkarılarak normalize puanlar bulunur.

> **Çözüm (derste verilmedi):** Normalize puanlar A için 0 ve 2, B için -2, -1 ve 1'dir. Profil ağırlıkları $w_A = (0 + 2)/2 = 1$, $w_B = (-2 - 1 + 1)/3 = -2/3$ olur.

Soru 2 (derste sözlü, 01:44:30): 9.2'deki fayda matrisinde A ile B'nin ve A ile C'nin Jaccard benzerliği nedir? Bu ölçü sezgiyi yakalar mı?

Derste verilen çözüm: A ile B'nin kesişimi 1 film, birleşimi 5 film: $1/5$. A ile C'nin kesişimi 2 film (Twilight, Star Wars 1), birleşimi 4 film: $2/4$. $sim(A, C) > sim(A, B)$ çıktığı için Jaccard sezgiyi yakalamaz; puanları hesaba katmaz.

Soru 3 (derste sözlü, 01:50:06): A kullanıcısı kaç filme puan vermiş, ortalaması nedir ve merkezlenmiş puanları ne olur?

Derste verilen çözüm: Üç filme puan vermiş. Toplam $4+5+1 = 10$, ortalama $10/3$. Merkezlenmiş puanlar $2/3$, $5/3$, $-7/3$. B için aynı yolla ortalama $14/3$ bulunur. Merkezlenmiş kosinüsle $sim(A, B) \approx 0{,}09$, $sim(A, C) \approx -0{,}56$.

Soru 4 (derste sözlü, 02:08:28): Öğe-öğe işbirlikçi filtrelemede, komşuluk değeri 2 iken 5. kullanıcı 1. filme kaç puan verirdi?

Derste verilen çözüm: 1. satıra en benzer, 5. kullanıcının puan verdiği iki satır 3. ($s_{13} = 0{,}41$, puan 2) ve 6. ($s_{16} = 0{,}56$, puan 3) satırlardır.

$$r_{15} = \frac{0{,}41 \cdot 2 + 0{,}56 \cdot 3}{0{,}41 + 0{,}56} \approx 2{,}6$$

Dersin Toparlaması

Bu derste öneri sistemlerine giriş yapıldı. Çevrimiçi mağazaların uzun kuyruk sayesinde sınırsız sayıda az popüler ürün sunabildiği, ama bu ürünleri bulmanın zorlaştığı ve bu yüzden kişiye özel öneriye ihtiyaç duyulduğu anlatıldı. Problem bir fayda fonksiyonu $u : C \times S \to R$ ve fayda matrisinin boş hücrelerini doldurma problemi olarak tanımlandı. Üç temel sorun ortaya kondu: bilinen puanları (açık ya da örtük) toplamak, bilinmeyen puanları tahmin etmek ve öneri başarısını ölçmek. Her iki algoritma ailesinin ortak zayıflığı olarak soğuk başlangıç sorunu vurgulandı.

Tahmin için iki yaklaşım görüldü. İçerik tabanlı öneri, öğe profillerinden (filmler için oyuncu, yönetmen, tür; metinler için TF-IDF ile seçilen kelimeler) kullanıcı profili çıkarır ve kosinüs benzerliğiyle karşılaştırır. Başka kullanıcılara ihtiyaç duymaz, ama aşırı uzmanlaşmaya düşer. İşbirlikçi filtreleme ise benzer kullanıcıların ya da benzer öğelerin puanlarından tahmin yapar. Benzerlik ölçüsü olarak Jaccard ve sıradan kosinüsün sezgiyi yakalayamadığı, merkezlenmiş

kosinüsün (Pearson korelasyonu) ise yakaladığı gösterildi. Öğe-öğe filtrelemede komşuluk değeriyle ağırlıklı ortalama alınarak bilinmeyen puan hesaplandı ve uygulamada öğe-öğe yaklaşımının daha iyi çalıştığı söylendi. Son olarak başarı, fayda matrisinden ayrılan test verisi üzerinde RMSE (ya da MAE, MAPE) ile ölçüldü.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.